FIXPROOF

FINAL PROJECT REPORT

AI-assisted sports media verification and digital evidence



FIELD	DETAIL
Repository	https://github.com/varunchauhan12/fixproof-muse-hackathon
Live frontend	https://fixproof-frontend-1097663016023.europe-west1.run.app
Live backend	https://fixproof-muse-hackathon-git-1097663016023.europe-west1.run.app
Document date	4th November 2026
Submission	Meta AI Hackathon

Prepared for technical review. Implementation details and measured claims should be verified against the current repository before publication.

Contents

- 1. Executive Summary
- 2. Problem Statement
- 3. Solution Overview
- 4. Target Users and Use Cases
- 5. Key Features and Interface
- 6. Detection Pipeline
- 7. System Architecture
- 8. Technology Stack
- 9. AI/ML Pipeline and Model Integration
- 10. End-to-End Workflows
- 11. Data, API and Storage Design
- 12. Evidence Packaging and Cryptographic Integrity
- 13. Deployment and Configuration
- 14. Testing and Validation
- 15. Limitations and Future Work
- 16. Reproduction and Setup Instructions
- 17. Asset and Finalization Checklist

1. Executive Summary

FixProof is an AI-assisted platform for sports media visual verification and digital evidence workflows. It helps investigators register reference footage, search suspect media for visually similar frames, review candidate findings, generate investigation reports, and package supported artifacts for integrity checks.

The detection system combines conventional computer vision and vector retrieval with targeted multimodal AI analysis. Reference and suspect frames are represented as MobileNetV3-Small embeddings and indexed/searched with FAISS. Entropy filtering and perceptual hashing help refine candidate matches. Meta Muse models provide optional visual verification and investigation and report synthesis where invoked by the workflow.

FixProof also includes evidence packaging components based on SHA-256 hashes and Ed25519 signatures, plus PDF generation and optional human-approved email workflows. These mechanisms can help detect changes to files covered by the verification design, but do not prove an allegation is true or guarantee legal admissibility.

The frontend is built with Next.js and React, and the backend uses Python/FastAPI. Supabase provides cloud persistence and object storage. The implementation and service configuration should be checked against the current repository before claiming particular deployment properties or benchmark values.

2. Problem Statement

2.1 Investigative challenge

Sports rights holders and media investigators may need to compare reference broadcasts with clips or other suspect media that appear on digital platforms. Manual workflows can require repeated searching, frame-by-frame comparison, documentation, and preparation of evidence artifacts.

Potential reuse may include cropping, resizing, compression, color changes, or other transformations that complicate straightforward pixel matching. At the same time, visual similarity can produce false positives—for example, when different scenes share similar colours, uniforms, or layouts.

2.2 Design goals

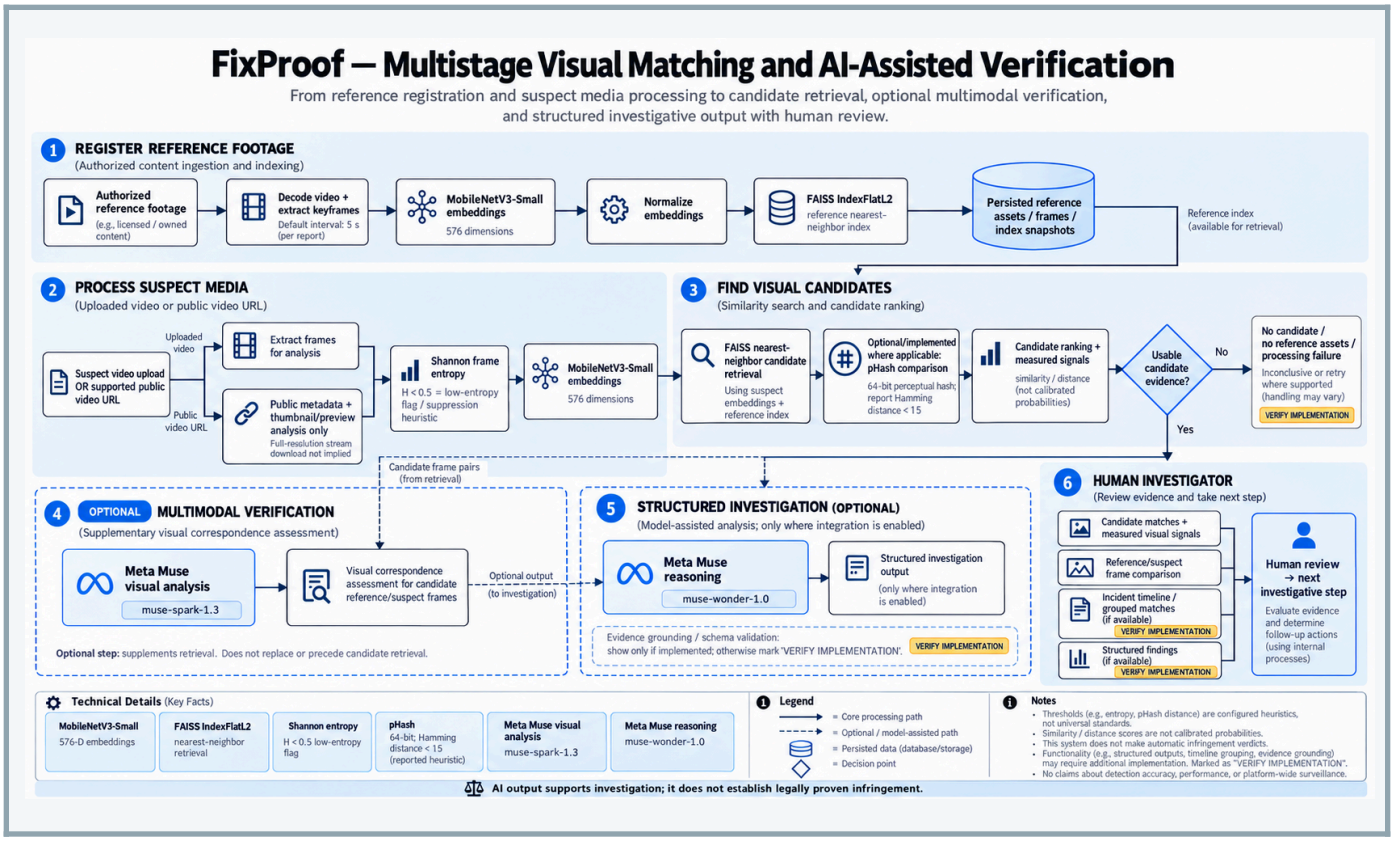
- Provide a searchable index of registered reference footage.
- Retrieve candidate frame matches using embedding-based similarity search.
- Use additional filtering and AI-assisted analysis where appropriate.
- Present findings for human review rather than automatically adjudicating infringement.
- Generate reports and evidence packages with integrity-checking mechanisms.

- Keep the limitations of public URL screening and model outputs explicit.

3. Solution Overview

FixProof organizes the investigation into a sequence of reference ingestion, frame representation, candidate retrieval, optional visual verification, structured reporting, and evidence export. Each stage has a distinct purpose: computer vision narrows the search space, while the AI models provide additional analysis and readable summaries.

- **Reference ingestion:** register a supported video asset and extract keyframes.
- **Visual representation:** generate 576-dimensional MobileNetV3-Small embeddings for extracted frames, as identified in the repository audit.
- **Candidate retrieval:** search the FAISS IndexFlatL2 index for nearest reference frames.
- **Candidate filtering:** apply entropy and perceptual-hash logic to help suppress uninformative frames and refine matches.
- **AI-assisted investigation:** invoke Meta Muse models for supported investigation tasks.
- **Evidence output:** generate a report and package supported artifacts for later verification.



4. Target Users and Use Cases

STAKEHOLDER	USE CASE	POTENTIAL VALUE
Broadcaster rights-protection teams	Register master/reference footage and investigate suspect clips.	Faster candidate discovery and organized investigation artifacts.
Sports leagues and rights holders	Maintain a searchable reference vault.	Reuse of indexed reference material across investigations.
Digital enforcement analysts	Review frame pairs, timelines, and similarity signals.	A more structured process for evaluating candidate matches.

STAKEHOLDER	USE CASE	POTENTIAL VALUE
Legal and compliance reviewers	Inspect investigation reports and integrity metadata.	Exportable artifacts and checks for covered file changes.
Creator partnerships teams	Prepare a human-reviewed licensing proposal where supported.	A non-takedown workflow option for further human consideration.

These are intended user groups and use cases. Actual operational suitability depends on deployment controls, data rights, model quality, and human review.

5. Key Features and Application Interface

5.1 Asset Vault and ingestion

The vault workflow accepts supported reference media, extracts keyframes, generates embeddings, and maintains an index for candidate retrieval. The supplied report describes a five-second keyframe interval as the default; confirm this setting in the current implementation before publishing it as a fixed behavior.

5.2 Threat Hunter and visual screening

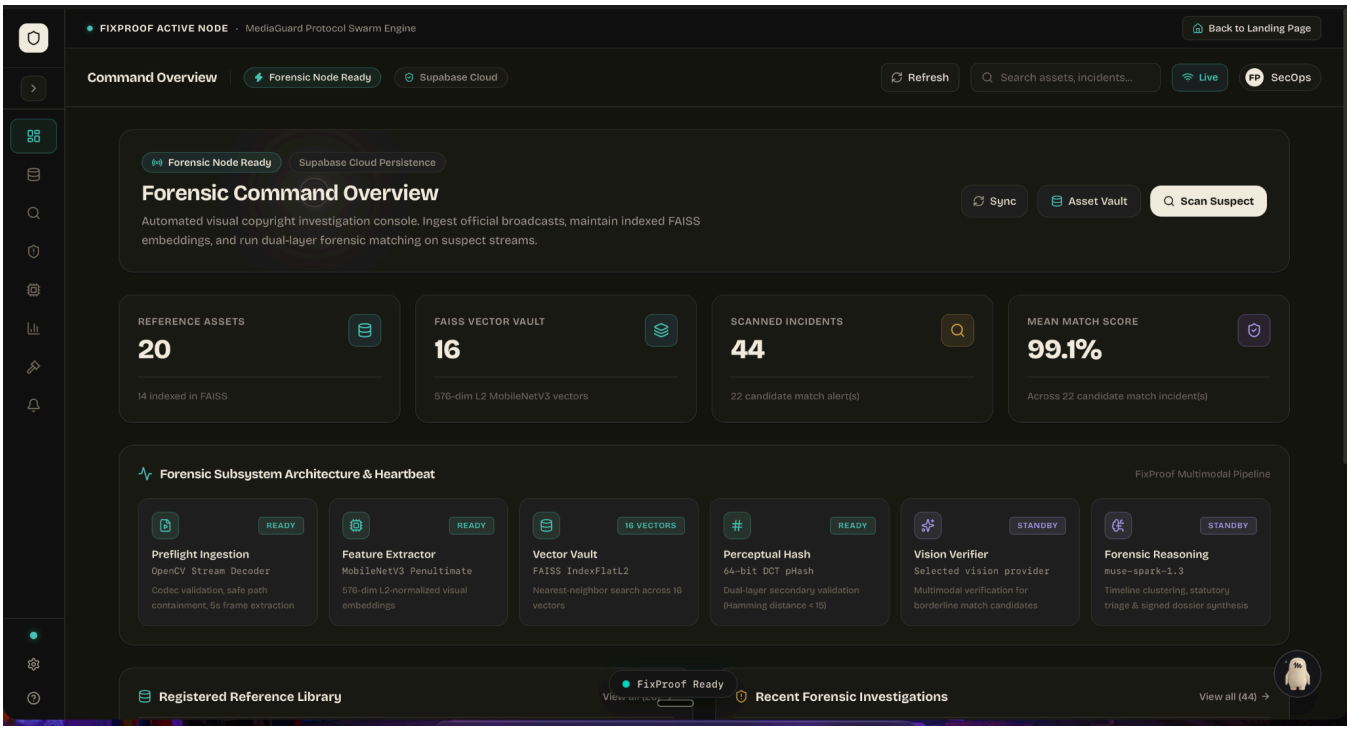
The platform supports uploaded-media scanning and a URL/preview screening path. According to the repository audit, public URL screening operates on accessible metadata and preview/thumbnail content; it does not bypass platform DRM or download full third-party streams.

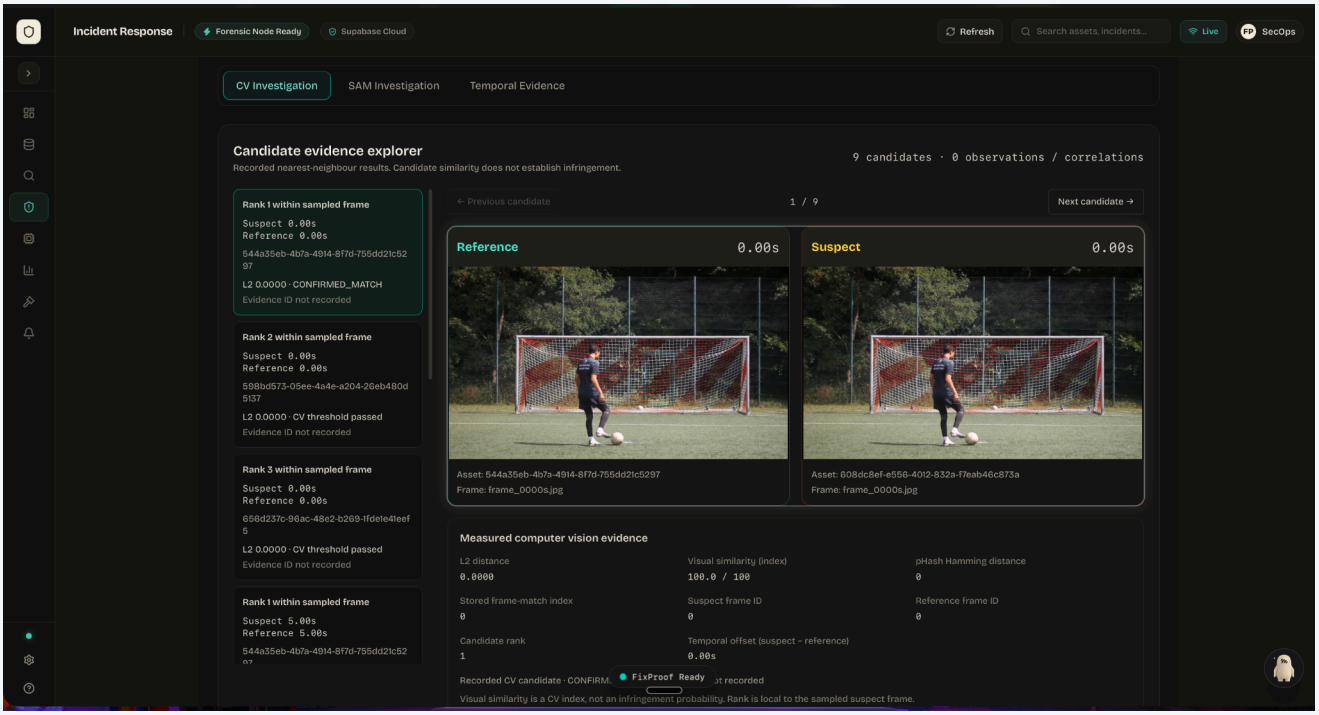
5.3 Investigation and reporting

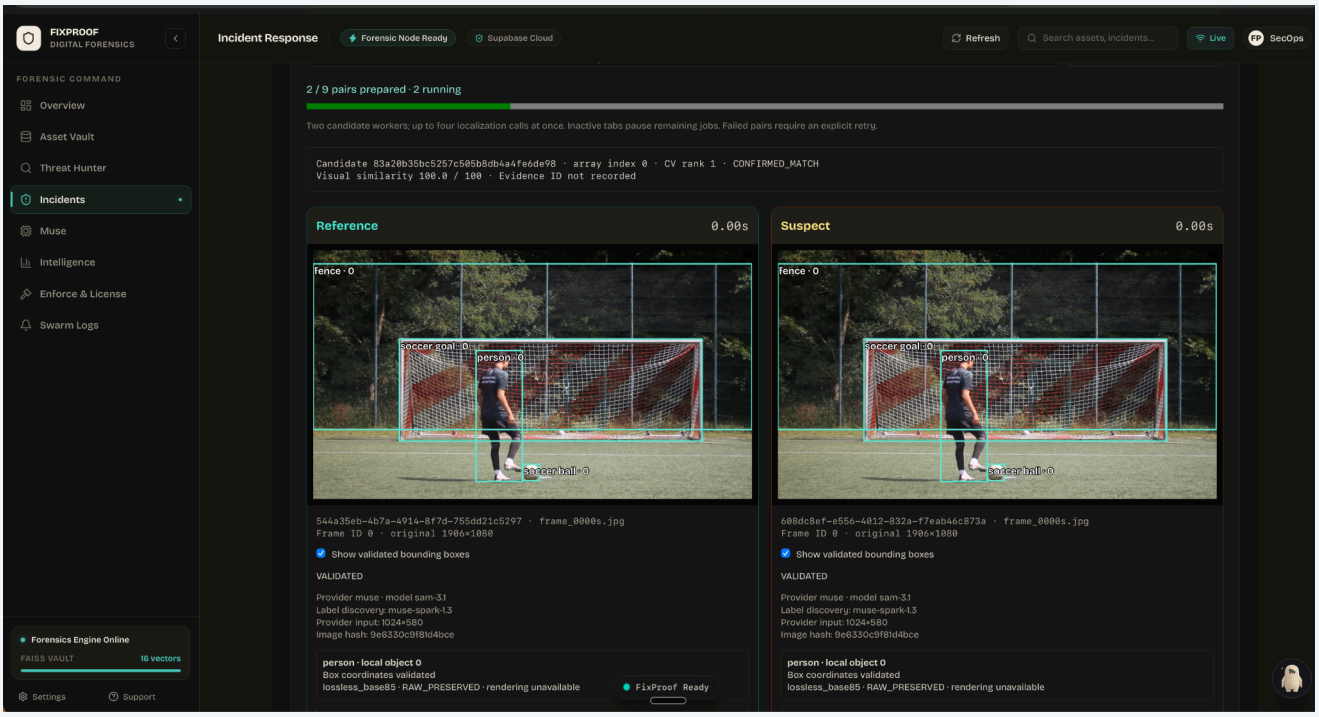
Investigators can inspect candidate incidents, request structured analysis, generate detailed report PDFs, draft supported notices or revenue-share proposals, and export evidence packages. External actions require the human-review/confirmation steps enforced by the implemented workflow.

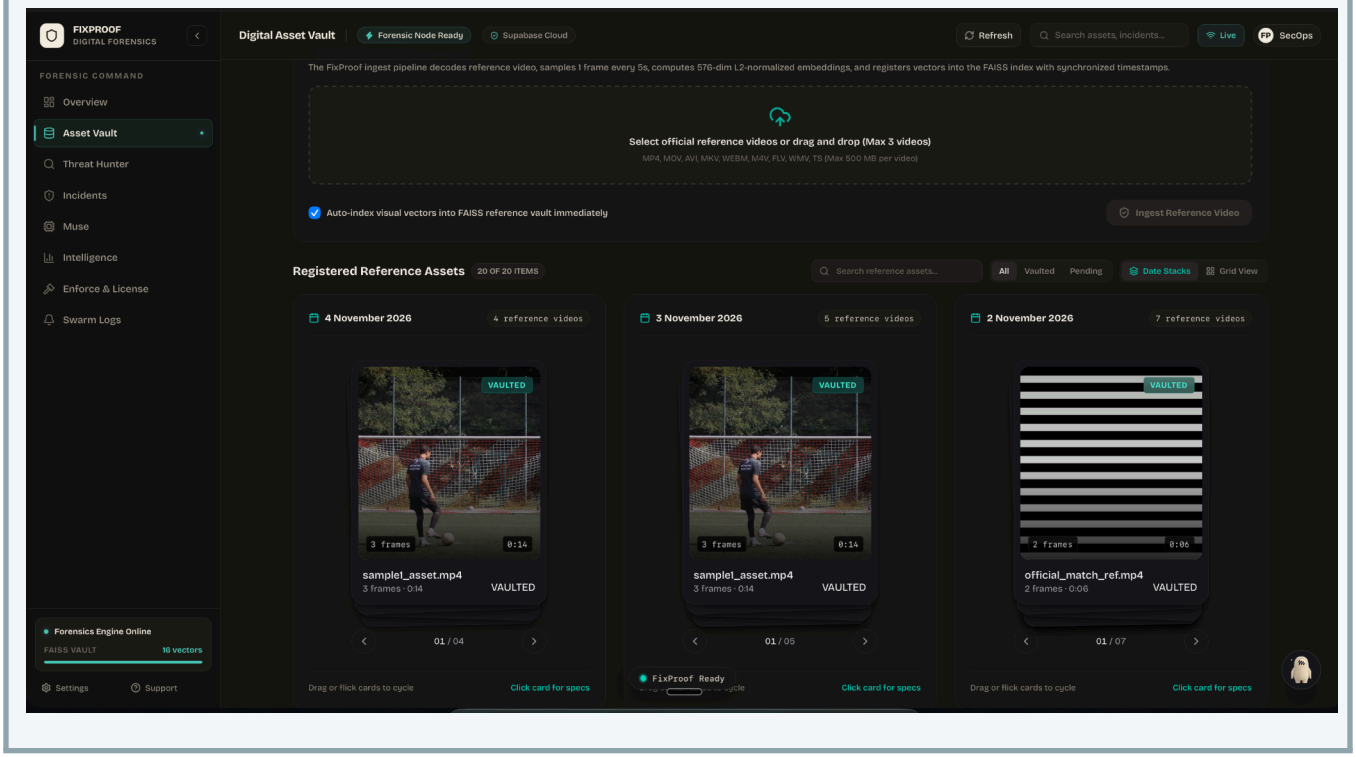
5.4 Product screenshots

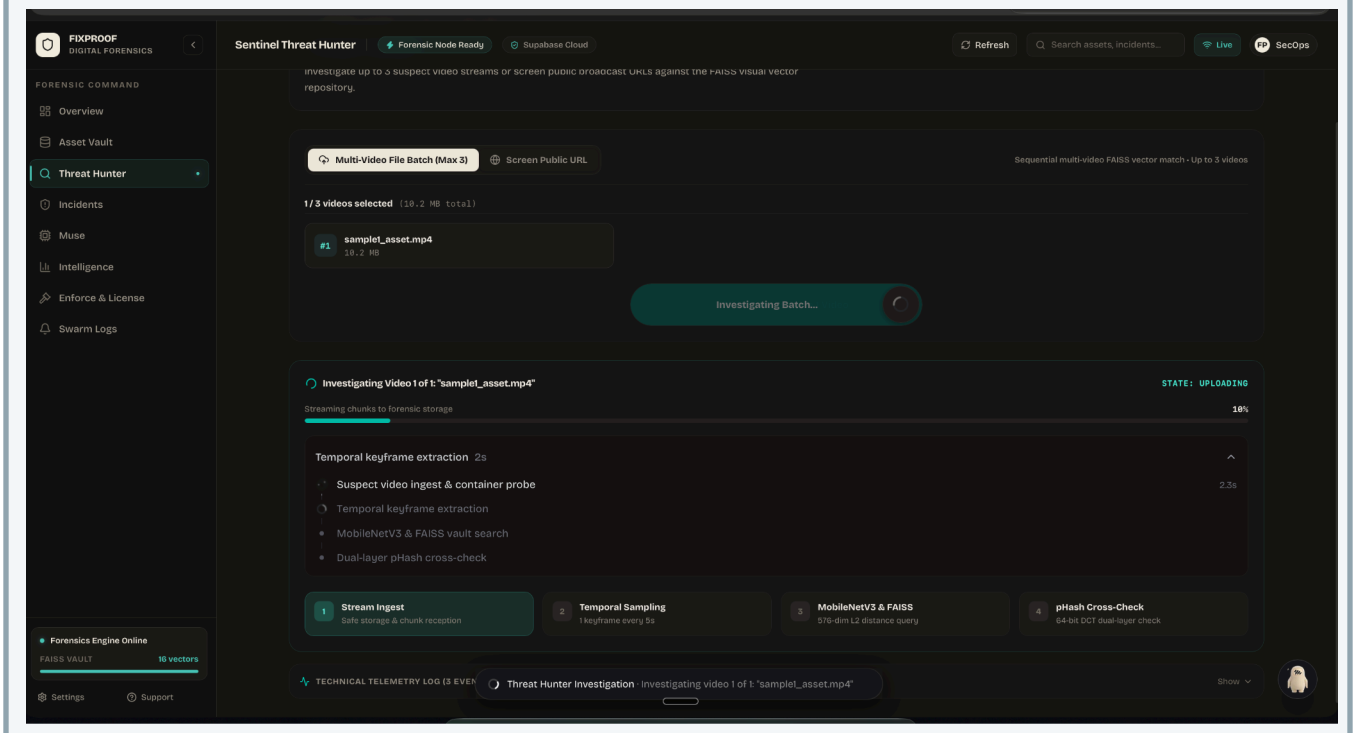
The source report referenced screenshots using local absolute paths. Replace those paths with repository-relative paths and commit the actual image files before publishing this report or README.




Incident Response
Forensic Node Ready
Supabase Cloud
Refresh
Search assets, incidents...
Live
SecOps
CV Investigation
SAM Investigation
Temporal Evidence
Candidate evidence explorer
Recorded nearest-neighbour results. Candidate similarity does not establish infringement.
9 candidates · 0 observations / correlations
Rank 1 within sampled frame
Suspect 0.00s
Reference 0.00s
544a35eb-4b7a-4914-8f7d-755dd21c5297
L2 0.0000 · CONFIRMED_MATCH
Evidence ID not recorded
Rank 2 within sampled frame
Suspect 0.00s
Reference 0.00s
588bd573-05ee-4a4e-a204-26eb480d5137
L2 0.0000 · CV threshold passed
Evidence ID not recorded
Rank 3 within sampled frame
Suspect 0.00s
Reference 0.00s
656d237c-96ac-48e2-b269-1fde1e41eef5
L2 0.0000 · CV threshold passed
Evidence ID not recorded
Rank 1 within sampled frame
Suspect 5.00s
Reference 5.00s
544a35eb-4b7a-4914-8f7d-755dd21c5297
← Previous candidate
1 / 9
Next candidate →
Reference
0.00s
Suspect
0.00s
Asset: 544a35eb-4b7a-4914-8f7d-755dd21c5297
Frame: frame_0000s.jpg
Asset: 608dc8ef-e556-4012-832a-f7eab46c873a
Frame: frame_0000s.jpg
Measured computer vision evidence
L2 distance
0.0000
Visual similarity (index)
100.0 / 100
pHash Hamming distance
0
Stored frame-match index
0
Suspect frame ID
0
Reference frame ID
0
Candidate rank
1
Temporal offset (suspect − reference)
0.00s
Recorded CV candidate · CONFIRM
FixProof Ready
Visual similarity is a CV index, not an infringement probability. Rank is local to the sampled suspect frame.


FIXPROOF
DIGITAL FORENSICS
Incident Response
Forensic Node Ready
Supabase Cloud
Refresh
Search assets, incidents...
Live
SecOps
FORENSIC COMMAND
Overview
Asset Vault
Threat Hunter
Incidents
Muse
Intelligence
Enforce & License
Swarm Logs
Forensics Engine Online
FAISS VAULT
16 vectors
Settings
Support
2 / 9 pairs prepared · 2 running
Two candidate workers; up to four localization calls at once. Inactive tabs pause remaining jobs. Failed pairs require an explicit retry.
Candidate 83a20b35bc5257c505b8db4a4fe6de98 · array index 0 · CV rank 1 · CONFIRMED_MATCH
Visual similarity 100.0 / 100 · Evidence ID not recorded
Reference
0.00s
Suspect
0.00s
fence · 0
soccer goal · 0
person · 0
soccer ball · 0
544a35eb-4b7a-4914-8f7d-755dd21c5297 · frame_0000s.jpg
Frame ID 0 · original 1906×1080
608dc8ef-e556-4012-832a-f7eab46c873a · frame_0000s.jpg
Frame ID 0 · original 1906×1080
Show validated bounding boxes
VALIDATED
Provider muse · model sam-3.1
Label discovery: muse-spark-1.3
Provider input: 1024×580
Image hash: 9e6330c9f81d4bce
person · local object 0
Box coordinates validated
lossless_base85 · RAW_PRESERVED · rendering unavailable
FixProof Ready





6. Detection Pipeline

6.1 Reference media processing

1. Accept a supported reference video through the reference-vault workflow.

2. Extract keyframes using the configured sampling interval.
3. Compute MobileNetV3-Small embeddings, reported as 576-dimensional float vectors.
4. Insert/reference the vectors in a FAISS IndexFlatL2 index alongside frame and asset metadata.
5. Persist the index and related metadata according to the configured storage mode.

6.2 Suspect media candidate search

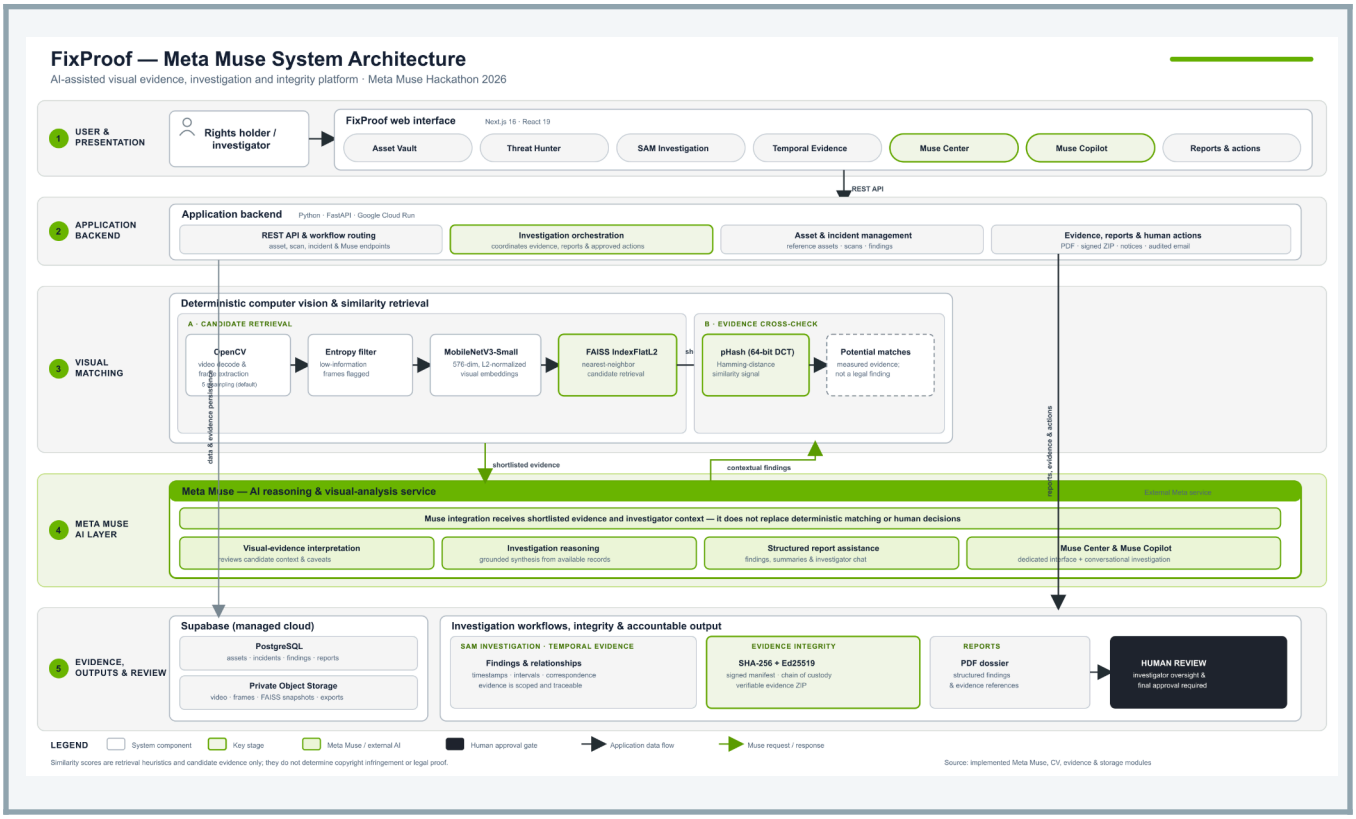
1. Process uploaded suspect media or the content accessible to the public URL screening path.
2. Filter low-information frames using Shannon entropy rules.
3. Query the reference-vector index to find nearest candidate frames.
4. Use pHash comparisons and configured thresholds to refine candidate status.
5. Create an incident/findings record for subsequent review and analysis.

6.3 Optional multimodal verification

Shortlisted frame pairs can be passed to Meta Muse vision (muse-spark-1.3 by default) in the implemented visual verification workflow. If that step is unavailable or inconclusive, the result should be treated as unverified rather than as a definitive match.

7. System Architecture

At a high level, the frontend communicates with the FastAPI backend. The backend coordinates video processing, similarity search, calls to the Meta Model API, database/object-storage access, report generation, and evidence export. The frontend should not call the model provider directly; provider credentials belong on the server side.



LAYER	COMPONENTS	RESPONSIBILITY
Frontend	Next.js, React, Bricolage Grotesque	Dashboard, asset ingestion, incidents, investigation, and evidence UI.

LAYER	COMPONENTS	RESPONSIBILITY
API/backend	Python, FastAPI, Uvicorn	REST endpoints and orchestration of processing/report/evidence workflows.
Visual matching	OpenCV, MobileNetV3-Small, FAISS, entropy, pHash	Frame extraction, representation, candidate retrieval, and filtering.
AI services	Meta Model API; Muse vision and reasoning models	Visual verification and structured/detailed analysis as invoked by workflows.
Persistence	Supabase PostgreSQL and Object Storage	Application records and supported media/index/artifact storage.
Outputs/integrity	ReportLab, ZIP, SHA-256, Ed25519, optional Resend	PDFs, evidence packages, integrity checks, and human-approved email workflows.

8. Technology Stack

AREA	TECHNOLOGY	PURPOSE
Frontend	Next.js, React, Tailwind CSS	Web application and responsive interface.
Typography / UI	Bricolage Grotesque; Lucide React, as used in source	Visual identity and interface components.
Backend	Python, FastAPI, Uvicorn	REST service and workflow orchestration.
Image/video processing	OpenCV, Pillow, imagehash	Frame extraction and perceptual image processing.
Visual embeddings	PyTorch / Torchvision MobileNetV3-Small	Generate frame feature vectors.
Vector search	FAISS CPU, IndexFlatL2	Nearest-neighbor candidate retrieval.
AI inference	OpenAI-compatible Python client to the Meta Model API	Call configured Meta Muse model endpoints.
Storage	Supabase PostgreSQL and Object Storage	Persist application data and supported artifacts.
Documents / integrity	ReportLab, cryptography, ZIP utilities	Generate PDFs, sign supported manifests, and package artifacts.
Email	Resend	Optional notification/document dispatch after human confirmation.

Package versions change and should be read from the current lockfiles/manifests. Avoid hard-coding specific dependency versions in published copy unless they have been checked against those files.

9. AI/ML Pipeline and Model Integration

9.1 MobileNetV3-Small embeddings

The repository audit identifies torchvision MobileNetV3-Small with the final classification layer replaced to produce a 576-dimensional feature vector. The implementation normalizes embeddings before indexing/search. These embeddings support candidate retrieval; they are not a semantic or legal judgment on their own.

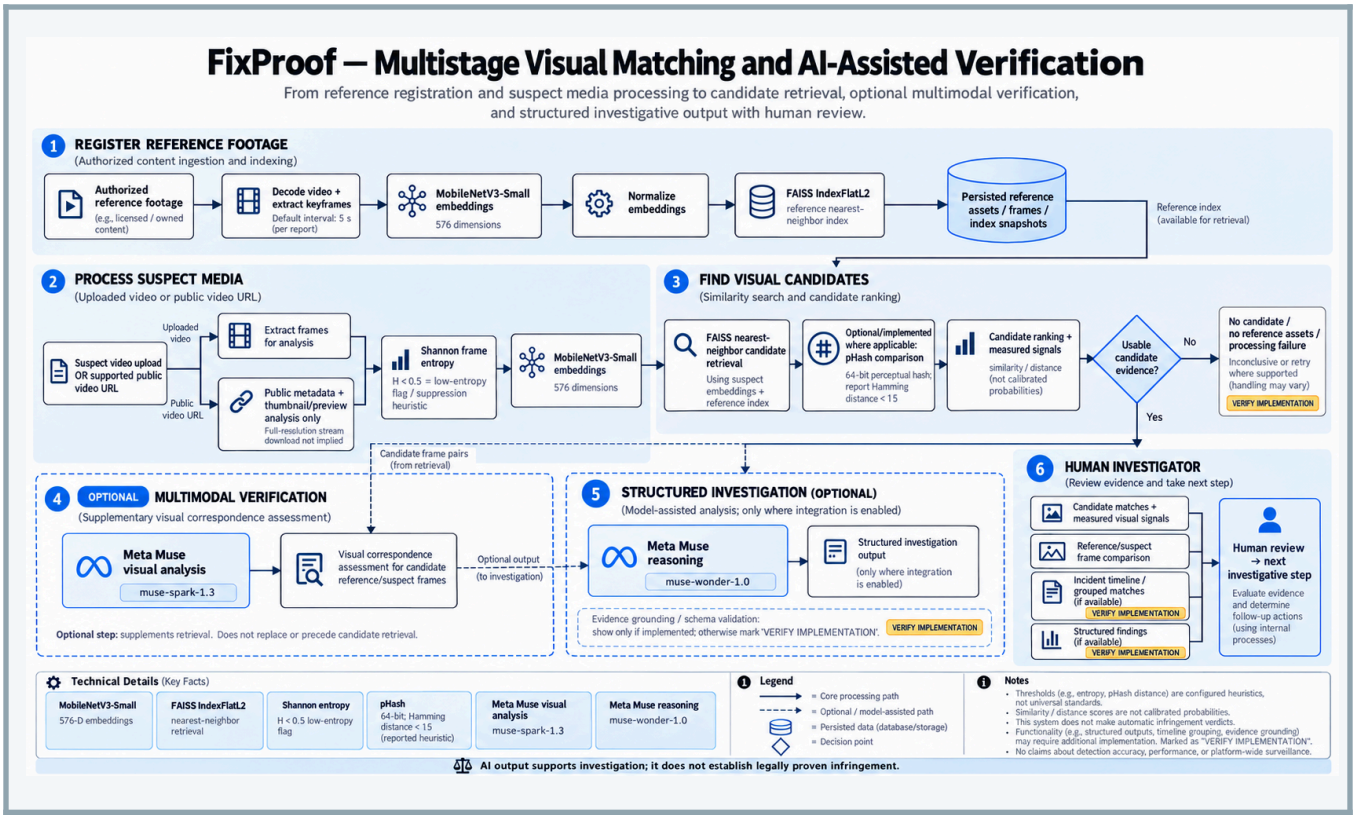
9.2 Shannon entropy filtering

Shannon entropy is used as a frame-information heuristic. Low-entropy frames may represent black frames, fades, or uniform imagery that can produce unhelpful matches. The reported H < 0.5 cutoff must be confirmed against the current matcher implementation and documented as a configured heuristic, not a universal threshold.

9.3 Meta Muse model roles

MODEL ID	DOCUMENTED ROLE	SOURCE CALL-SITES REPORTED
`muse-wonder-1.0`	Structured investigation analysis and supported notice/document drafting.	`backend/app/providers/meta.py`; investigation and document workflows.
`muse-spark-1.3`	Pairwise visual verification of shortlisted reference and suspect frames.	`backend/app/providers/meta.py`; visual verification workflow where invoked.
`muse-wonder-1.0`	Detailed report synthesis using the configured Muse reasoning model.	`backend/app/detailed_report.py`.

The model IDs shown are repository defaults and can be overridden by deployment configuration. Verify the active configuration before reporting deployed behavior. Do not imply every model is invoked in every scan.



9.4 Grounding and safeguards

- The system prompt directs the model to act as an evidence analyst, not a legal judge.
- Where implemented, report validation checks evidence IDs against the supplied evidence/interval identifiers.
- AI output can be incomplete or incorrect; ambiguous results require human review.
- Internal reasoning text should not be exposed to clients. Confirm sanitization behavior in the current source before making broad privacy claims.

10. End-to-End Workflows

10.1 Register reference footage

1. Upload a supported reference asset through the vault API/UI.
2. Extract keyframes and compute frame embeddings and hashes.
3. Update the FAISS index and associated frame/asset metadata.
4. Persist data according to the configured local/cloud storage mode.

10.2 Scan suspect media

1. Submit supported media or a URL to the appropriate screening workflow.
2. Extract accessible frames or previews.
3. Apply entropy filtering, vector retrieval, and pHash candidate refinement.
4. Review the created incident and its candidate findings.

10.3 Investigate and export

1. Invoke visual verification and/or structured analysis where supported.
2. Review the model output and available technical evidence.
3. Generate the applicable report and evidence package.
4. Verify the package and inspect the returned status.
5. Only after a human reviews and confirms the action should optional external email dispatch occur.

11. Data, API and Storage Design

11.1 Main data entities

ENTITY / AREA	PURPOSE
Reference assets and frames	Store reference asset metadata and extracted frame information.
Reference vectors / FAISS index	Link frame metadata to searchable vector representations.
Incidents	Store scan results and candidate-match summaries.
Investigation reports	Store structured report outputs and associated incident/model metadata.
Vault snapshots	Track saved index snapshots and related metadata, where enabled.
Vision verifications	Cache supported pairwise visual verification results, where enabled.
Evidence packages	Record supported manifest, digest, and signature metadata.

The table is a high-level description. Confirm exact table names, columns, foreign keys, bucket names, and row-level security policy from current migrations and Supabase configuration before publishing a detailed schema.

11.2 Key API endpoints

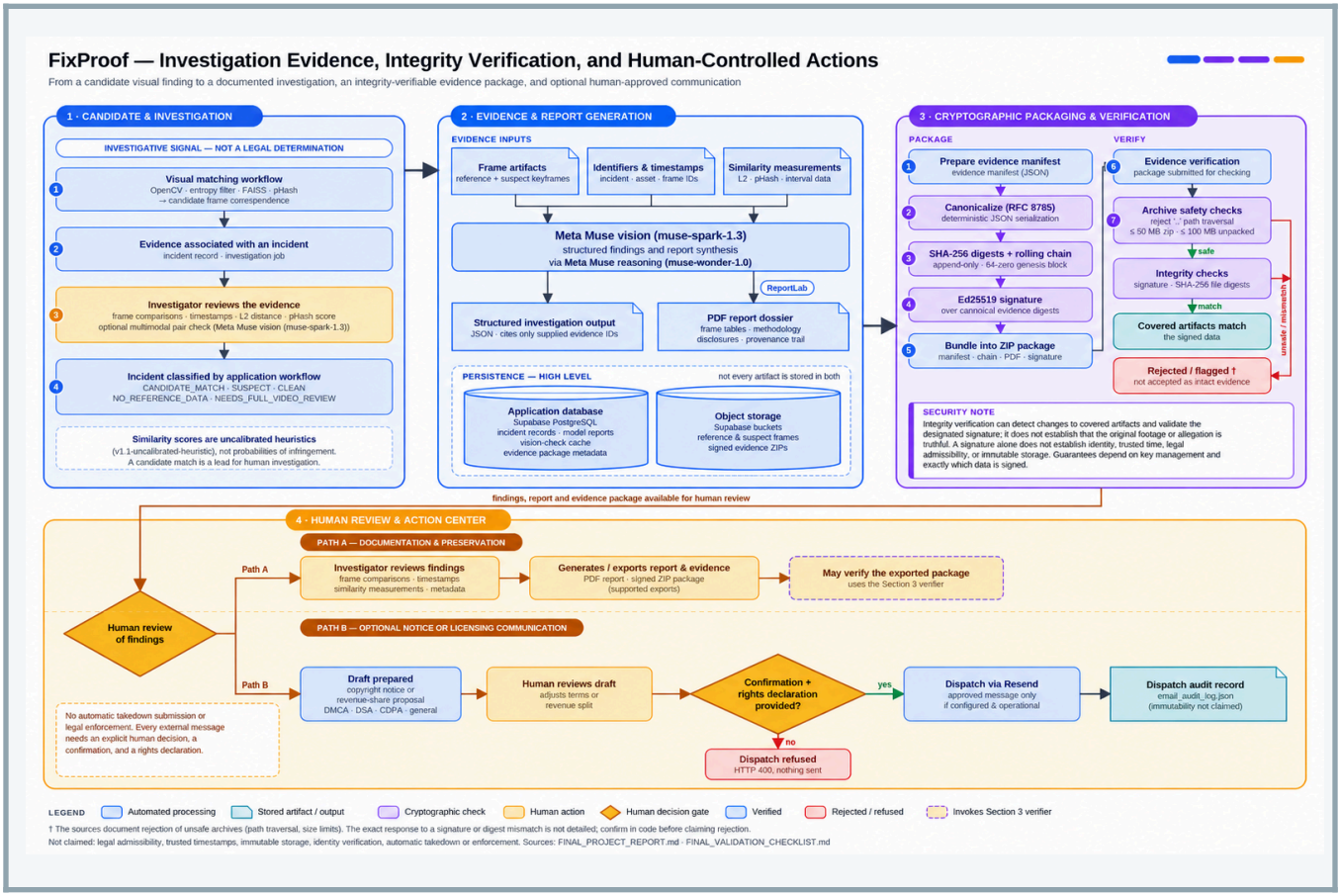
METHOD	ENDPOINT	PURPOSE
GET	`/health`	Basic service health.
GET	`/api/vault/status`	Vault status and snapshot/mode information.
GET	`/api/muse/status`	AI provider/model status information.

METHOD	ENDPOINT	PURPOSE
POST	`/api/vault/ingest`	Ingest/register a reference asset.
GET	`/api/vault/assets`	List registered reference assets.
POST	`/api/threats/scan`	Scan uploaded suspect media.
POST	`/api/threats/screen-url`	Screen accessible public URL/preview content.
GET	`/api/incidents`	List recorded incidents.
POST	`/api/threats/{incident_id}/investigate`	Generate structured incident analysis.
POST	`/api/muse/detailed-report`	Generate a detailed report.
GET	`/api/muse/detailed-report/{report_id}/pdf`	Download a generated PDF.
POST	`/api/muse/draft-document`	Draft a supported notice or proposal.
GET	`/api/threats/{incident_id}/reports/export`	Export a signed evidence package.
POST	`/api/threats/reports/verify-package`	Verify an evidence package.

12. Evidence Packaging and Cryptographic Integrity

FixProof supports generation of evidence artifacts and ZIP packages, with cryptographic integrity checks over the parts covered by its manifest/signing implementation. SHA-256 produces file digests; Ed25519 provides digital signatures for supported signed data; canonicalization helps produce deterministic manifest bytes where used.

1. Collect the relevant report and supported investigation artifacts.
2. Build the manifest containing the metadata covered by the package design.
3. Canonicalize the manifest if RFC 8785 canonicalization is implemented for that code path.
4. Compute the designated SHA-256 digests and rolling chain, if configured by the implementation.
5. Sign the data covered by the Ed25519 signing procedure.
6. Assemble the signed files and manifest in a ZIP package.
7. Run the verifier to check supported signatures, digests, and archive-safety conditions.



Interpretation of cryptographic results

A valid digest or signature supports integrity/authentication for the specific data covered by the scheme. It does not prove that the media was lawfully obtained, that an allegation is true, or that a court will admit the evidence.

13. Deployment and Configuration

The project report identifies a deployed Google Cloud Run backend and a live frontend. The following URLs were provided for the project:

Frontend: https://fixproof-frontend-1097663016023.europe-west1.run.app

Backend: https://fixproof-muse-hackathon-git-1097663016023.europe-west1.run.app

A prior report states that `/health` returned HTTP 200. Refresh the health check close to submission and include the check timestamp if documenting it as a current status.

13.1 Environment variables

VARIABLE	PURPOSE / NOTES
`FIXPROOF_DB_MODE`	Selects local or Supabase persistence mode.
`SUPABASE_DB_URL`	PostgreSQL connection string for cloud persistence.
`SUPABASE_URL`	Supabase project/service endpoint.
`SUPABASE_SERVICE_ROLE_KEY`	Privileged backend storage access; keep secret.
`MODEL_API_KEY`	Authenticates requests to the Meta Model API; keep secret.

VARIABLE	PURPOSE / NOTES
`MUSE_REASONING_MODEL`	Optional override for the Muse reasoning model.
`MUSE_VISION_MODEL`	Optional override for the Muse vision model.
`FIXPROOF_SIGNING_KEY_HEX`	Signing-key seed/configuration for evidence package signatures; keep secret.
`RESEND_API_KEY`	Optional email dispatch credential; keep secret.
`FROM_EMAIL`	Outgoing message sender address.
`NEXT_PUBLIC_API_URL`	Frontend base URL for the backend API.
`PORT`	Runtime port binding in the hosting environment.

Treat the list as a configuration checklist rather than a universal set of required variables; exact requirements depend on storage mode and enabled features. Never put real values in documentation or commit them to Git.

14. Testing and Validation

14.1 Checks reported as executed

CHECK	COMMAND / SCOPE	REPORTED RESULT
Frontend production build	`cd frontend && npm run build`	Passed; compiled successfully; `/`, `/_not-found`, and `/dashboard` routes generated.
Backend core unit tests	`cd backend && .venv/bin/python -m unittest tests.test_cloud_runtime tests.test_vault`	25 tests ran; `OK`; 1 skipped; no failures or errors.
Matcher test limitation	`tests.test_matcher` in a clean cloud database without seeded fixtures	Two reported assertion failures because expected status differs when no valid reference data is present.

These results were supplied by an Antigravity repository audit. They cover the specified commands only and have not been independently rerun while preparing this Word document.

14.2 Additional results requiring confirmation

- Earlier project documentation reports 163 frontend integration checks. Confirm the exact test commands and latest output before publishing this count.
- Confirm the current `/health`, `/api/vault/status`, and `/api/muse/status` responses if listing deployment checks.
- Confirm live Meta Muse inference and end-to-end evidence verification separately from unit tests.
- Capture the commit hash and date associated with published test results.

15. Limitations and Future Work

15.1 Current limitations

- Public URL screening is limited to accessible metadata, thumbnails, and preview content. It does not bypass DRM or establish access to full third-party live streams.
- Audio fingerprinting is listed as future work in the supplied project report; do not present it as an active detection layer.
- Matching thresholds and similarity scores are heuristics and should not be interpreted as calibrated probabilities of infringement.
- Model responses may be incomplete or wrong and must be reviewed by a person.
- Some tests require seeded reference assets or an appropriate mock/local database configuration.

- Cryptographic checks only cover the data actually included in the signature/manifest procedure. They do not guarantee legal admissibility or immutable storage.

15.2 Future work

- Explore audio fingerprinting after establishing a validated evaluation dataset and clear licensing constraints.
- Benchmark visual retrieval and model-inference latency under reproducible hardware and dataset conditions.
- Add/validate supported platform API integrations where permissions and terms allow.
- Improve seeded integration test fixtures so matcher tests can run deterministically in isolated environments.
- Expand model/output evaluation for false positives, false negatives, and human-review consistency.

16. Reproduction and Setup Instructions

The commands below follow the repository structure and setup commands reported in the supplied audit. Verify them from a clean checkout before claiming clean-install reproducibility.

16.1 Clone the repository

```
git clone https://github.com/varunchauhan12/fixproof-muse-hackathon.git
cd fixproof-muse-hackathon
```

16.2 Backend

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
# Configure required environment variables locally
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

16.3 Frontend

```
cd frontend
npm install
npm run dev
```

The frontend development server is reported at `http://localhost:3000`. Configure `NEXT_PUBLIC_API_URL` and any required backend variables in a local untracked environment file. Do not commit keys, tokens, service-role credentials, or private signing keys.

17. Asset and Finalization Checklist

Use this checklist to finish the Word report and its repository-ready companion files. The placeholders in this document are intentional and should be replaced with real files before final publication.

ASSET	ACTION	SUGGESTED PATH	PRIORITY
Logo	Insert FixProof logo at the cover placeholder; optional if the cover is intentionally text-only.	docs/assets/fixproof-logo.png	Optional
Detection pipeline	Insert the final detection pipeline image in Section 3.	docs/diagrams/detection-pipeline.png	Recommended
System architecture	Insert the final system architecture image in Section 7.	docs/diagrams/system-architecture.png	Recommended
Dashboard screenshot	Insert a real dashboard screenshot in Section 5.	docs/screenshots/dashboard_overview_expanded.png	Recommended
Incident screenshot	Insert a real candidate matching / incident screen.	docs/screenshots/dashboard_incidents_view.png	Recommended
AI/report screenshot	Insert a real Muse analysis or report screen.	docs/screenshots/dashboard_muse_view.png	Recommended

ASSET	ACTION	SUGGESTED PATH	PRIORITY
Vault upload screenshot	Insert real vault upload screen if it adds value.	docs/screenshots/dashboard_vault_upload.png	Optional
Threat scan screenshot	Insert real scan screen if it adds value.	docs/screenshots/dashboard_threat_hunter_active.png	Optional
Evidence lifecycle	Insert the final evidence lifecycle image in Section 12.	docs/diagrams/evidence-lifecycle.png	Recommended
AI model routing	Skip unless it adds meaningful detail beyond the architecture diagram.	docs/diagrams/ai-model-routing.png	Optional